

Claude Code can consume, transmit, and compromise your .env files even if you tell it not to

Cite as: Martin Paul Eve, "Claude Code can consume, transmit, and compromise your .env files even if you tell it not to", *eve.gd: Martin Paul Eve*, April 19, 2026, <https://doi.org/10.59348/m47sp-w0777>.

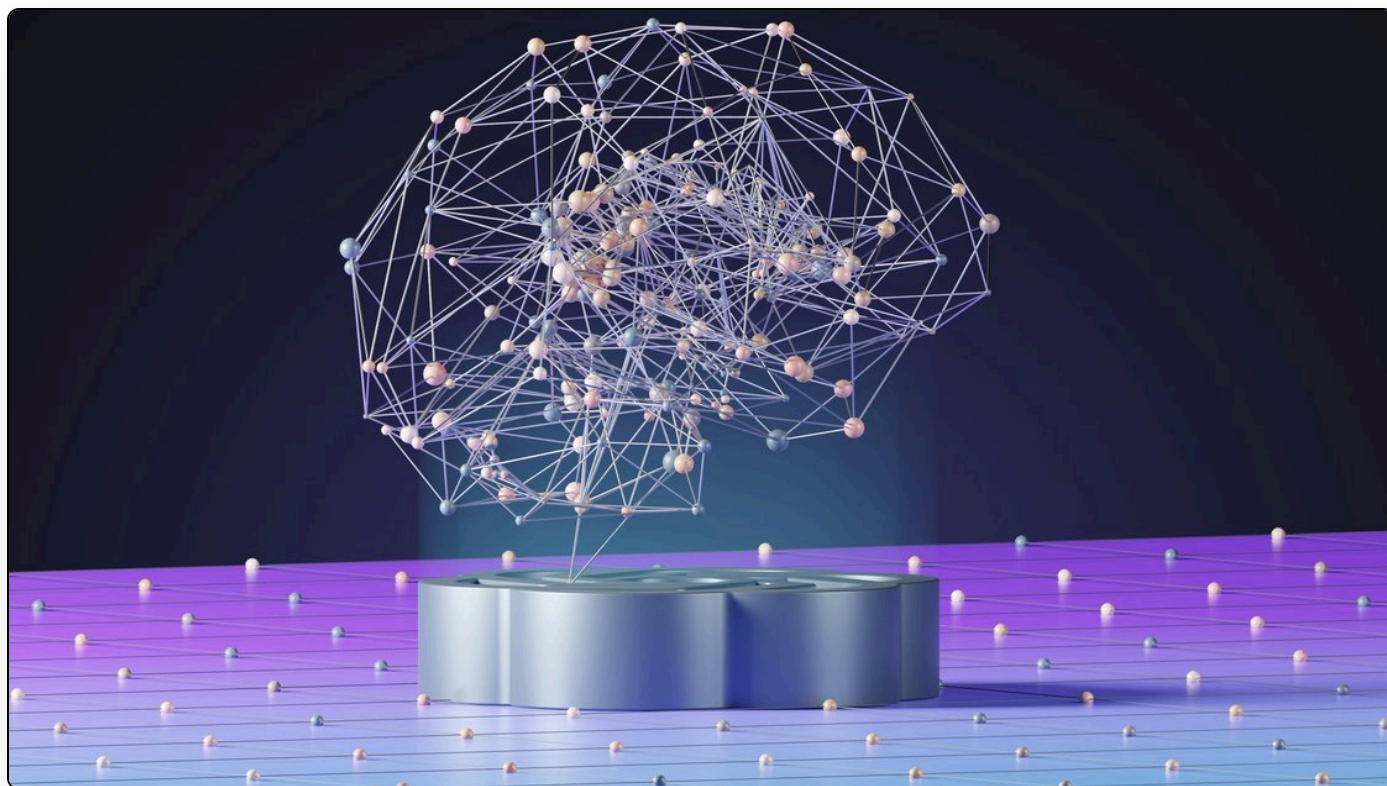


Image: An abstract image representing AI (Growtika on Unsplash)

My colleague Ian discovered the other day that, alarmingly, even if you tell Claude code not ever to read your .env files, it may still do so and send the result back to its servers, thereby compromising your local development secrets. Ian is using Claude via cursor, but his AGENTS.md file specifically instructed Claude not to read this file. It did so anyway.

The “12 Factor App” paradigm tells us that we must store configuration in the environment. But when developing locally, this means that we need some way of bootstrapping the environment... and .env files are the most common way to do this. Whack your config in a .env file, then, just before the app loads, load the file into the container environment.

This creates some serious security problems, of course. Every experienced developer has a gitignore file template that blocks the commit of .env files. But it's simple, convenient, and works.

The other thing about this paradigm though is that of course in an ideal world all of the configuration secrets used on a development machine would be sandboxed development credentials for external services. If you're doing development work against an external API, you should not be using the production secret on your local development machine. But this is totally naive. Smaller, custom production APIs do not necessarily have or provide sandboxed test modes. Mocking such services locally is a huge drain on time, and one also cannot guarantee that one has properly mocked all the edge cases for testing. In short, it is highly possible that .env files in local development circumstances can contain live API keys and other sensitive data. Sure, they should not, but we do not live in an ideal world.

Claude code, obviously, works by sending responses to and from their server, which runs inference on the context it is provided. If Claude reads the .env file, this will be transmitted back to Anthropic. This could then be incorporated into future training runs. And it could then be possible for a user to extract these data from the model in future. This could lead to credential compromise.

There are many suggested ways of blocking Claude from accessing this file. I have heard suggestion of a .claudeignore file, but believe this is not implemented. Obviously, we have tried putting the ignore instruction in CLAUDE.md and AGENTS.md. Another colleague suggested that Linux or Mac file permissions could be set so that Claude simply could not access the .env file at all (though this could then create permissions problems for running the application in test mode; indeed, I would be worried about the complexity of the file access situation here and having to run Claude in a different user account space to isolate it, which would impose severe restrictions on the coder's ability). There is an official "[deny rules](#)" mechanism that one is apparently supposed to use, but Claude could circumvent this by writing a custom script or pipe chain.

The way I will handle this in my setup is by using 1Password environments. This software lets you replace variables in a .env file with vault secrets. 1Password then mounts a virtual .env file, with the secrets, in the location you specify. This file is never actually written to disk but all requests to

access the file trigger authorisation requests - so, in my setup, I will have to enter my YubiKey and touch the flashing light on it to confirm physical presence. For more, see the [1Password documentation on environments](#).

With this setup, there will be a separation of concerns. If Claude wants to run the debug server, then that application can be given permission to see the virtual .env file. Likewise, running tests could get permission from me to use secrets. However, if Claude is just scanning the directory for files and I see a popup asking to use the .env file, I will deny such permission. Certainly, there could still be confusion. What if Claude wants to launch the application and then the application requests permission for the file? I could become confused and give permission when I am actually giving it to a sub agent. However, this is the best I have come up with for now on a balance between security and practicality or comfort.

I cannot tell whether we are being overly cautious or underly careful. However, my personal belief is that the guardrails employed by Claude here are not sufficient. And there should be a stronger set of mechanisms to deny access to sensitive files.